objects sent to `processGrid`. It turns out this isn't a difficult mathematical problem. We take the square root of the number of datapoints and round it up to the nearest whole number to get the right number of rows and columns for our grid. This makes sense when you think about how a grid is a set of rows and columns that allows you to place a cell on one of those rows and columns for each datapoint. The number of rows times columns needs to be at least the number of cells (the number of datapoints). If we decide to have the same number of rows as columns, then it's that number squared.

Listing 10.2 Updated `processGrid` function

```
function processGrid(data) {
   var rows = Math.ceil(Math.sqrt(data.length));
   var columns = rows;

   var cell = 0;

   for (var i = 0; i < rows; i++) {
      for (var j = 0; j < columns; j++) {
         if (data[cell]) {
             data[cell].x = j;
             data[cell].y = i;
             cell++;
         }
         else {
           break;
         }
      }
   }

   return data;
}
```

Calculates the number of rows/columns

Initializes a variable to walk through the dataset

Loops through the rows and columns

This assumes the data consists of an array of objects.

Sets the current datapoint to corresponding row and column

Increments the datapoint variable

To test our nascent grid layout, we can load tweets.json and pass it to grid. The grid function displays the graphical elements onscreen based on their computed grid position. In the following listing, you can see how we'd pass data from tweets.json to our grid layout.

Listing 10.3 Using our grid layout

```
d3.json("tweets.json", function(error, data) {
})

function makeAGrid(data) {
   var scale = d3.scale.linear().domain([0,5]).range([100,400]);
   var grid = d3.layout.grid();
   var griddedData = grid(data.tweets);
   d3.select("svg").selectAll("circle")
      .data(griddedData)
      .enter()
      .append("circle")
      .attr("cx", function(d) {return scale(d.x);})
```

A scale to fit our grid onto our SVG canvas

Sets circles to a scaled position based on the layout's calculated x and y values